CPSC 375 High-Performance Computing

Lecture 10

Sockets

Practices

Supported Services: TCP

TCP (Transmission Control Protocol) *service:*

- *connection-oriented:* setup required between client and server processes
- *reliable transport* between sending and receiving process
- *flow control:* sender won't overwhelm receiver
- *congestion control:* throttle sender when network overloaded
- *does not provide:* timing, minimum throughput guarantee, security
- e.g., HTTP, FTP, SSH, SMTP

Supported Services: UDP

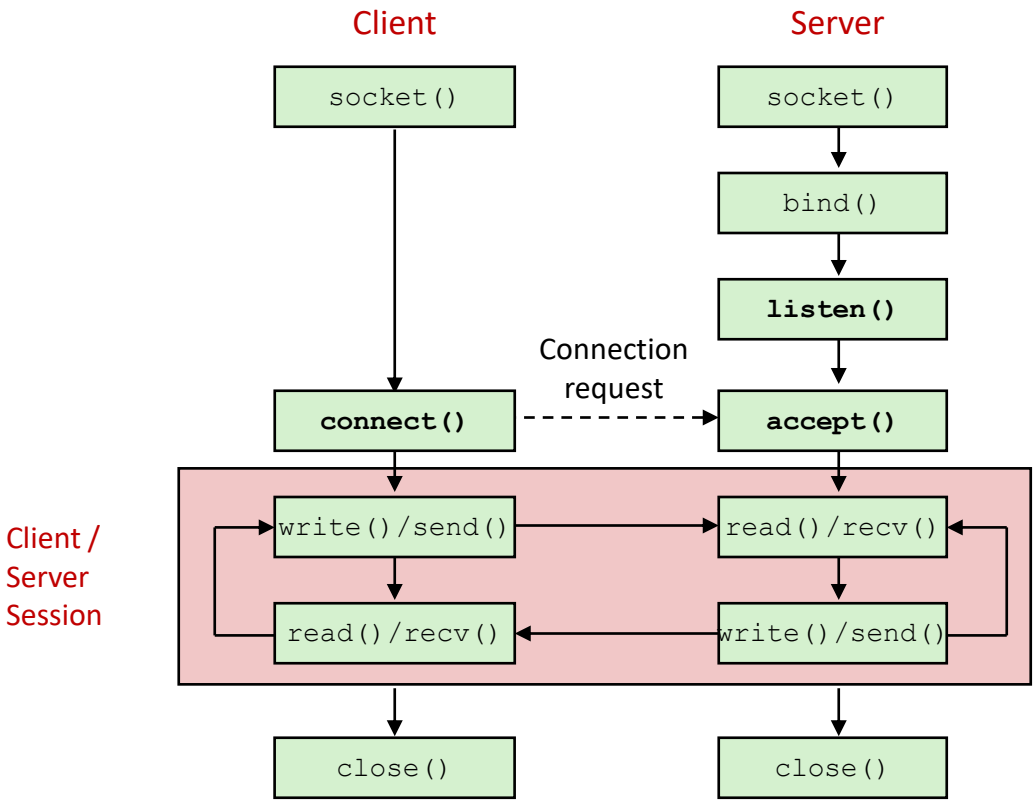
UDP (User Diagram Protocol) *service:*

- *connectionless:* no handshaking before the two processes start to communicate.
- *unreliable data transfer* between sending and receiving process
- *does not provide:* reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup,

Q: Why bother? Why is there a UDP?

- commonly used for real-time applications:
  - audio/video streaming, multiplayer video games, etc.
  - *DNS* for resolving domain names
  - apps that don't require a response from the other end, e.g., IP *broadcasting* or *multicasting*

TCP Client and Server



Socket Header Files

```
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
```

socket()

```
int socket(int family, int type, int protocol);
```

creates a socket and returns a socket descriptor or -1 on error, where

- `family = AF_INET` (IPv4) or `AF_UNIX` (IPC)
- `type = SOCK_STREAM` (for TCP) or `SOCK_DGRAM` (for UDP)
- `protocol = 0` (system default)

Important: always check return values; on error, print an error message by calling `perror()`. If it's a fatal error, call `exit()` to terminate the program.

Unix domain TCP socket (server)

Create a TCP socket:

```
sockfd = socket(AF_UNIX, SOCK_STREAM, 0);
```

Bind a TCP socket:

```
int bind(int sockfd, struct sockaddr *addr,
         int addrlen);
```

assigns the address specified by `addr` to the socket referred to by the file descriptor `sockfd`, where

- `addr` points to the address structure `struct sockaddr`
- `addrlen = sizeof(struct sockaddr)`

struct sockaddr

```
struct sockaddr {
    unsigned short sa_family;
    char           sa_data[14];
};
```

Important: make sure to set all the socket structures with null values before assigning any values (`bzero()` or `memset()`)

struct sockaddr

In practice, we use:

```
struct sockaddr_un {  // for Unix sockets
    sa_family_t sun_family;          /* AF_UNIX */
    char        sun_path[108];       /* Pathname */
};
```

listen()

```
int listen(int sockfd,int backlog);
```

marks `sockfd` as a socket that will be used to accept incoming connection requests, where

- `sockfd =` socket descriptor of the server
- `backlog =` the maximum number of connections allowed on this socket

accept()

```
int accept(int sockfd, struct sockaddr *cliaddr,
socklen_t *addrlen);
```

- extracts the first connection request on the queue of pending connections for the listening socket, `sockfd`,
- creates a new connected socket, and
- returns a new file descriptor referring to that socket, where
  - `sockfd` = socket descriptor of the server
  - `cliaddr` = points to `struct sockaddr` of the client (to be filled by the function)
  - `addrlen` = `sizeof(struct sockaddr)`

read()

```
int read(int fd, void *buf, int len)
```

- reads up to count bytes from file and place into buffer, where
  - `fd`: file descriptor
  - `buf`: pointer to array
  - `len`: number of bytes to read
- returns number of bytes read or -1 if error

write()

```
int write(int fd, void *buf, int len)
```

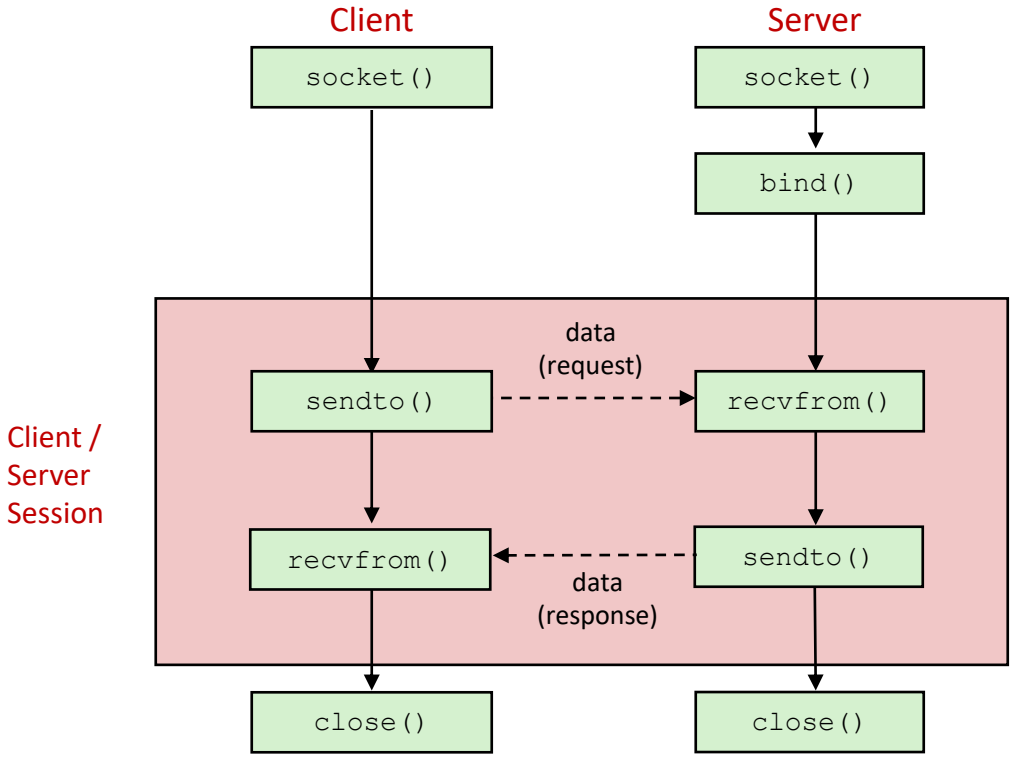
- writes `len` bytes from buffer to file, where
  - `fd`: file descriptor
  - `buf`: pointer to array
  - `len`: number of bytes to write
- returns number of bytes written or -1 if error

close()

```
int close(int sockfd)
```

- closes an open socket, where
  - `sockfd`: file descriptor
- returns zero on success. On error, -1 is returned, and `errno` is set to indicate the error.

UDP Client and Server



Unix Domain UDP Socket

Create a socket:

```
sockfd = socket(AF_UNIX, SOCK_DGRAM, 0);
```

sendto()

```
int sendto(int sockfd, const void *msg, int len,
           unsigned int flags, const struct sockaddr *to,
           int tolen);
```

where

- `sockfd` = socket descriptor
- `msg` = points the data to send
- `len` = the number of bytes to send
- `flags` = 0
- `to` = points to `struct sockaddr` for the receiver
- `tolen` = `sizeof(struct sockaddr)`

recvfrom()

```
int recvfrom(int sockfd, void *buf, int len,
             unsigned int flags, struct sockaddr *from,
             int *fromlen);
```

where

- `sockfd` = socket descriptor
- `buf` = points the buffer to read the data into
- `len` = the number of bytes to receive
- `flags` = 0
- `from` = points to `struct sockaddr` for the sender (to be filled by the function)
- `fromlen` = `sizeof(struct sockaddr)`

recv()

```
int recv(int sockfd, void *buf, int len,
         unsigned int flags);
```

- reads up to count bytes from socket and place into buffer, where
  - `sockfd`: file descriptor
  - `buf`: pointer to array
  - `len`: number of bytes to receive
  - `flags`: set it to 0 (equivalent to `read()`)
- returns the number of bytes received or -1 if error

send()

```
int send(int sockfd, const void *buf, int len,
         int flags);
```

- writes `len` bytes from buffer to file, where
  - `sockfd`: socket descriptor
  - `buf`: pointer to array
  - `len`: number of bytes to write
  - `flags`: set it to 0 (equivalent to `write()`)
- returns the number of bytes written or -1 if error

From Hostname to IP Address: `gethostbyname()`

```
struct hostent *gethostbyname(const char *hostname);
```

returns a structure of type `hostent` for the given `hostname`, where

```
struct hostent {
    char  *h_name;       /* official name of host */
    char **h_aliases;    /* alias list */
    int    h_addrtype;   /* host address type */
    int    h_length;     /* length of address */
    char **h_addr_list;  /* list of addresses */
}
```

Here, the first entry of `h_addr_list` has an IP address for `hostname`.

Network Addresses

- A host is uniquely identified by its IP address.
- An IP address is a combination of four 8-bit numbers (0~255)
- e.g.,

```
turing.cs.trincoll.edu 157.252.16.12
lab3.cs.trincoll.edu   157.252.16.15
```

- Network addresses uses a canonical byte order convention known as *network byte order*, where
  - the most significant byte is at the smallest memory address and the least significant byte at the largest (*big-endian*)
- Requires conversion between two different byte orders.

Byte Order Functions

- for converting data between a host's internal representation and network byte order

`htons()`	host to network `short`
`htonl()`	host to network `long`
`ntohl()`	network to host `long`
`ntohs()`	network to host `short`

- e.g., to convert a port number (16 bit or `short`) to network byte order:

`htons(`*`portnum`*`)`

Useful IP Address Functions

```
int inet_aton(const char *strptr, struct in_addr
*addrptr)
```

converts `strptr` in the Internet standard dot notation to a network address `addrptr`.

```
in_addr_t inet_addr(const char *strptr)
```

converts `strptr` in the Internet standard dot notation to an integer value suitable for use as an Internet address.

```
char *inet_ntoa(struct in_addr inaddr)
```

converts the specified Internet host address `inaddr` to a string in the Internet standard dot notation.

Demo

- Unix socket of TCP type for IPC
  - `server1.c` and `client1.c`
- Internet socket of TCP type on localhost
  - `server2.c` and `client2.c`
- Internet socket of TCP type for two hosts
  - `server3.c` and `client3.c`
- Unix socket of UDP type for IPC
  - `server4.c` and `client4.c`